

# LC 3016: Minimum Number of Pushes to Type Word - II

## Concept & Visual Logic

|      |     |      |
|------|-----|------|
| 1    | abc | def  |
| ghi  | jkl | mno  |
| pqrs | tuv | wxyz |
| *    | 0   | #    |

placed at first pos → **1 push**

second pos → **2 push**

third pos → **3 push**

last pos → **4 push**

**hence only 8 useful buttons**

## Algorithm Steps

**S1** sort the string by descending frequency

**S2** Eg "aaaaaaddddy"

then

a : 6

d : 3

y : 2

**S3**

utmost priority is to keep a as first as possible so it uses least amount of push

## Code Implementation

```
class Solution {
public:
    int minimumPushes(string word) {
        int ans=0;
        int n=word.size();

        if(n>0){
            ans+=min(n,8); //eg 5 aur 25 , 5 he to 5 keys me pehle rkh
do                                //25 hua to , pehle 8 , bach gaye 17
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(2*n,16); //17 me se , 16 taps pehle 8 me for 8
more char                                //bach gaye ab 9 char
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(3*n,24); //max ab 24 aur tap me 8 character aur
jaayenge                                //bach gaye ab 9-8=1 char
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(4*n,32); //just 4 more taps for last char
            n-=min(n,8);      //no more char left

        }

        return ans;
    }
};
```